ROJECT REPORT

Project Title:

School Management System

Data Structures & Algorithms

Submitted By:

Muhammad Nouman Khan 24-SE-104

Muhammad Arslan Fareed 24-SE-36

Muhammad Saqlain 24-SE-102

1. ABSTRACT

This project is a console-based software application developed in C++ to automate the administrative activities of an educational institution. The system moves beyond simple file storage by implementing a Multi-Class Architecture, allowing the management of five distinct class sections simultaneously. It utilizes core Data Structures concepts-specifically Structures (structs), Static Arrays, and Matrices-to handle complex relationships between students, attendance, and academic results. The system is optimized for performance using Binary Search (O(log N)) and One-Pass Aggregation (O(N)), ensuring instant data retrieval and analytics.

2. SYSTEM ARCHITECTURE (THE DESIGN)

The system is built on a Hierarchical Data Model, meaning data is organized in a "Parent-Child" relationship rather than a flat list.

1. **The Global Level (School):** The application manages a global array of 5 Classroom objects.
2. **The Class Level:** Each Classroom object acts as an independent container. It holds its own unique counter (studentCount) and its own array of students.
3. **The Student Level:** Inside the class, each Student is a complex record (struct) containing:
    - **Identity:** Roll Number (Integer), Name (String).
    - **Academics:** An array of 5 floating-point numbers representing marks.
    - **Attendance:** An array of 30 integers representing the daily status (-1, 0, 1).

This design ensures strict Data Isolation. Operations performed on "Class 1" do not affect "Class 2," mimicking real-world physical registers.

3. MODULE BREAKDOWN & EXPLANATION

Student Management

This module handles the Creation, Reading, Updating, and Deletion of records.

- Sorted Insertion Strategy: Unlike basic systems that append data to the end of a list, this system finds the correct numerical position for a new Roll Number and inserts it there.
    - Why? It keeps the array permanently sorted.
    - *Benefit:* A sorted array allows us to use high-speed Binary Search for all future lookups.
- Destructive Deletion: When a student is deleted, the system performs a "Left Shift" operation. It overwrites the deleted record by moving all subsequent records one step backward. This ensures there are no "holes" or empty gaps in the memory.

Attendance System

This module replaces the tedious task of manual roll calls.

- Batch Processing: The system iterates through the active student array loop. It displays the student's name and waits for a simple binary input (1 for Present, 0 for Absent).
- Data Structure: Conceptually, this creates a Boolean Matrix (Rows = Students, Columns = Days). The system can generate reports showing total percentage

of the students, distinguishing between days where attendance was not taken (Value -1) and days the student was absent (Value 0).

Academic Results

- Marks Processing: The system allows entry of marks for 5 distinct subjects.
- The Gazette Report: This is a feature designed for teachers. It prints a detailed table (Gazette) for the entire class. It calculates the Total, Average, and Letter Grade for every student dynamically at the time of printing. This means if a mark is updated, the grade updates automatically without needing a separate calculation step.

Analytics Engine

This module provides "Business Intelligence" for the class.

- Performance Metrics: Instead of running three separate loops to find the Topper (Max), Lowest Scorer (Min), and Class Average, the system uses a Single-Pass Algorithm. It traverses the array once, updating the Min, Max, and Total accumulators simultaneously. This makes the analytics extremely fast and efficient (O(N) complexity).

4. ALGORITHMS & TECHNICAL CONCEPTS

This section details the computer science principles applied in the code.

4.1. Binary Search (Optimization)

The system uses Binary Search for all lookup operations (Searching, Modifying, Marking Attendance).

- Logic: It compares the target Roll Number with the middle element of the array. If the target is smaller, it discards the upper half; if larger, it discards the lower half.
- Impact: For a dataset of 1,000 students, a Linear Search might take 1,000 steps. Binary Search takes only ~10 steps. This demonstrates a focus on Time Complexity optimization.

4.2. Insertion Sort Logic

To maintain the sorted order required for Binary Search, the addStudent function uses insertion logic.

- Logic: It identifies the index i where the new student belongs. It then runs a loop from the last element down to i, shifting every element one position to the right (index+1 = index) to create a vacant slot. The new data is then placed in this slot.

4.3. Defensive Programming (Input Validation)

To prevent runtime crashes, a custom validation layer (getValidInt) acts as a firewall between the user and the logic.

- Mechanism: It intercepts every input. If the input stream enters a "fail state" (e.g., user types text into an integer field), the function clears the error flags, flushes the memory buffer, and reprompts the user. This ensures the software is Robust and Crash-Resistant.

5. SYSTEM LIMITATIONS & FUTURE SCOPE

Limitations

- Static Memory: The system uses a fixed array size (60 students). This is a design choice for memory safety but limits scalability.
- Volatile Data: As a console-based logic demonstration, data is stored in RAM and resets when the program closes.

Future Scope

- File Persistence: Implementing File I/O to save records to .csv or .txt files would make the system usable for long-term data storage.
- Dynamic Structures: Replacing arrays with Linked Lists would allow the system to handle an unlimited number of students.

6. CONCLUSION

The School Management System is a comprehensive demonstration of applying theoretical Data Structures to practical problems. By leveraging Arrays for storage and Binary Search for retrieval, the system achieves a balance of speed and simplicity. The implementation of Multi-Class support and Automated Analytics proves that even simple console applications can solve complex administrative challenges when engineered correctly.